Superscalar RISC-V Processor Core

Technical Architecture & Implementation Report

Author: Vekariya Pritkumar
Affiliation: Indian Institute of Technology, Ropar

---

1 Project Overview

This document details the architectural design, RTL implementation, and verification of a high-performance **Superscalar RISC-V Processor Core**. The design addresses the bottleneck of single-issue architectures by executing multiple instructions concurrently per clock cycle, directly increasing the Instruction Per Cycle (IPC) throughput.

The processor is modeled in **SystemVerilog** and verified using **Xilinx Vivado** for dynamic functional analysis and static timing evaluation.

2 Core Architectural Features

- **Parallel Execution Engine:** Dual/multi-issue instruction execution pipelines running simultaneously.
- **Dynamic Instruction Scheduling:** Integrated advanced scheduling concepts to allow out-of-order execution and reduce pipeline hazard stalls.
- **Cycle-Accurate Verification:** Fully simulated and validated across all 5 standard pipeline stages.

3 Pipeline Stages & System Architecture

The core expands upon the traditional 5-stage RISC-V pipeline framework to support concurrent instruction pathways:

3.1 1. Fetch (IF) & Decode (ID)

- **Fetch Stage:** Retrieves multiple 32-bit instructions per cycle from instruction memory.
- **Decode Stage:** Parses operands, resolves immediate values, and performs dependency checks to ensure structural and data hazards are identified early.

3.2 2. Advanced Scheduling & Out-of-Order Execution (EX)

- Implements instruction window buffer logic to inspect instruction readiness.
- Dispatches non-conflicting instructions to parallel Arithmetic Logic Units (ALUs).
- Minimizes structural pipeline stalls through dynamic scheduling mechanisms.

3.3 3. Memory (MEM) & Writeback (WB)

- **Memory Stage:** Handles data load and store transactions while enforcing strict memory access ordering.
- **Writeback Stage:** Commits register results back to the Architectural Register File in cycle-accurate sequence.

4 Tools & Implementation

Table 1: Design Specifications and Toolchain

Category	Specification / Tool
Hardware Description Language	SystemVerilog
Target Architecture	RISC-V (32-bit Integer Base ISA)
Synthesis & Simulation Environment	Xilinx Vivado EDA Suite
Verification Paradigm	RTL Testbenches & Behavioral Simulation
Design Metric Focus	Instruction Per Cycle (IPC) & Pipeline Throughput

5 Simulation & Results

The core underwent cycle-accurate simulation in Xilinx Vivado. The custom testbenches verified structural integrity across all pipeline stages during complex execution sequences involving data hazards and conditional branches.

5.1 Key Findings

1. **IPC Optimization:** Improved overall Instruction Per Cycle execution speed compared to the single-issue baseline.
2. **Hazard Mitigation:** Out-of-order execution scheduling successfully reduced stall cycles during dependency-heavy instruction streams.
3. **Cycle Accuracy:** Verified functional correctness across all 5 pipeline stages (Fetch, Decode, Execute, Memory, Writeback).

6 Repository Setup & Build Instructions

To run behavioral simulations locally:

1. Open Xilinx Vivado and create a new RTL project.
2. Add all SystemVerilog source files (`.sv`) under the `rtl/` directory.
3. Set `tb_superscalar_core.sv` as the top-level simulation module.
4. Run **Behavioral Simulation** to observe cycle-accurate stage transitions.